Real-time Monitoring System for Elderly Posture Analysis Based on CNN

Focusing on Lightweight and Optimized Deep learning models for Edge Device Deployment

CNN 기반 노인 자세 분석을 통한 실시간 모니터링 시스템

엣지 디바이스 적용을 위한 딥러닝 모델 경량화 및 최적화 중심

요 약

해당 시스템은 딥러닝 기술을 활용하여 노인의 자세 및 보행을 분석하고, 이상 징후를 조기에 감지하여 질병 예방에 기여하는 것을 목표로 한다. 이를 위해 OpenCV 와 TensorFlow 를 활용하여 노인의 움직임 데이터를 수집하고 분석하며, 치매, 관절염 등의 질병을 조기 발견할 수 있도록 지원하는 시스템을 구축한다. 또한, Django 프레임워크와 Python 을 기반으로 한 웹 애플리케이션을 개발하며, HTML 과 CSS 를 사용하여 사용자 친화적인 인터페이스를 제공하는 웹사이트를 제작할 예정이다. 제안된 시스템은 Admin 과 User 라는 두 엔터티에서 액세스하며, Admin 은 먼저 유효한 로그인 자격 증명으로 로그인해야 한다.

20203163 홍형준

Department of AI Convergence

Graduate School of Soongsil University

TABLE OF CONTENTS

CHAPTER 1 서론 ····· 1

1.1 **연구 배경 및 필요성** ····· 1

1.2 **문제 정의** ····· 1

1.3 **연구 목적 및 범위** ····· 1

1.4 **논문의 구성** ····· 2

1.5 **기대 효과** ····· 2

CHAPTER 2 관련 연구 ····· 2

2.1 **기존 낙상/자세 감지 시스템** ····· 2

2.2 **객체 탐지 모델 비교** ····· 2

2.3 **모델 최적화 기법** ····· 3

CHAPTER 3 시스템 설계 ····· 3

3.1 **전체 시스템 아키텍처** ····· 3

3.2 하드웨어 및 소프트웨어 구성 ····· 4

3.3 웹 어플리케이션 아키텍처 ····· 4

3.4 데이터베이스 설계 ····· 5

3.5 Web 서버와 Raspberry Pi 간 연동 구조 ····· 5

3.6 데이터 흐름 및 파이프라인 ····· 6

3.7 사용자 인터페이스 설계 ····· 7

CHAPTER 4 모델 경량화 및 최적화 ····· 7

4.1 기존 낙상/자세 감지 시스템 ····· 8

4.1.1 YOLOv8n 학습 방법 ····· 8

4.2 최적화 기법 적용 ····· 9

4.2.1 Pruning ····· 10

4.2.2 TFLite 변환 ····· 12

4.2.3 Quantization ····· 13

4.3 최적화 전후 비교 ····· 14

CHAPTER 5 실험 및 결과 분석 ····· 14

5.1 **실험 환경** ····· 14

5.2 **실험 시나리오** ····· 15

5.3 **정확도, 추론 속도, 메모리 사용량** ····· 16

5.4 **엣지 디바이스 실시간 테스트 결과** ····· 17

CHAPTER 6 결론 및 향후 연구 ····· 18

6.1 **연구 요약** ····· 18

6.2 **한계점** ····· 18

REFERENCES ····· 20

APPENDICES ····· 20

Ⅰ. 서론

1.1 연구 배경 및 필요성

우리나라는 급속한 고령화를 경험하고 있으며, 이에 따라 노인 안전사고, 특히 낙상 문제가 심각한 사회적 이슈로 대두되고 있다. 2023 년 노인실태조사에 따르면 독거노인의 비중이 32.8%로 2020 년 대비 13.0%p 증가하였으며, 독거노인은 다른 가구형태에 비해 건강상태가 열악하고 우울증상과 생활상의 어려움이 더 큰 것으로 나타났다. 2024 년 질병관리청 통계에 따르면 응급실 내원 손상환자 중 70 세 이상이 19.3%로 가장 많았으며, 전체 손상에서 낙상이 40.0%로 가장 높은 비중을 차지했다.

낙상은 단순한 부상을 넘어 생명을 위협하는 심각한 문제이다. 낙상 경험이 시간에 따라 증가할수록 사망위험률이 약 15-17 배까지 높아지는 것으로 나타났으며, 낙상 경험자의 14.9%가 골절상을 입었다. 특히 독거노인의 경우 낙상 발생 시 즉각적인 도움을 받기 어려워 골든타임을 놓칠 위험이 크다. 국내 재가노인의 16.3-37.5%가 낙상을 경험하고 있으며, 발생빈도와 의료비가 매년 증가하는 추세이다.

전통적인 낙상 예방 방법은 환경 개선, 운동 프로그램, 약물 관리 등 예방적 접근에 한정되어 있으며, 실시간 위험 감지와 즉각 대응에는 한계가 있다. 웨어러블 센서 기반 시스템은 착용 불편함과 관리의 어려움으로 실용성이 떨어진다. 따라서 비침습적이면서도 실시간으로 낙상 위험을 감지할 수 있는 컴퓨터 비전 기반 모니터링 시스템의 필요성이 대두되고 있다.

1.2 문제 정의

센서 기반 시스템은 착용자의 불편함과 높은 비용, 설치 복잡도 등의 문제가 있으며, 영상 기반 시스템은 처리 속도와 정확도 간의 균형이 관건이다. 특히 엣지 디바이스에서 실시간 처리를 가능하게 하기 위해선 모델 경량화와 최적화가 필수적이다.

1.3 연구 목적 및 범위

본 연구는 YOLOv8 기반 CNN 모델을 활용하여 노인의 자세를 실시간으로 분석하고 낙상 위험을 조기 감지하는 엣지 디바이스 기반 모니터링 시스템을 개발하는 것을 목적으로 한다.

구체적인 연구 목표는 다음과 같다. 첫째, 노인의 일상 자세와 위험 자세를 포함하는 특화 데이터셋을 구축한다. 둘째, 엣지 디바이스에서 실시간 추론이 가능한 경량화 CNN 모델을 개발한다. 셋째, 키포인트 기반 낙상 위험도 평가 알고리즘을 개발한다. 넷째, Raspberry Pi 에서 최적화 및 배포한다. 다섯째, 실시간 알림 시스템을 구축한다.

연구 범위는 실내 단일 인물 자세 추정, RGB 카메라 기반 시스템, 저비용 엣지 디바이스 플랫폼으로 한정하며, 다중 인물 추적과 실외 환경은 제외한다.

본 연구의 차별점은 노인 특화 접근, 엣지 우선 설계, 실용성 중심, 프라이버시 보호, 오픈소스 지향에 있다.

1.4 논문의 구성

본 논문은 총 6 장으로 구성되며, 2 장에서는 관련 연구를 분석하고, 3 장에서는 제안 시스템의 구조를 설명한다. 4 장에서는 모델 경량화 및 최적화 기법을, 5 장에서는 실험과 그 결과를 다룬다. 마지막으로 6 장에서는 결론 및 향후 연구 방향을 제시한다.

1.5 기대 효과

본 연구는 학술적으로 노인 자세 분석 기반 마련과 엣지 AI 최적화 방법론을 제시하고, 사회적으로 노인 안전 증진, 독거노인 돌봄 지원, 의료비 절감에 기여하며, 산업적으로 저비용 케어 솔루션과 스마트 홈 케어 시장 활성화를 기대할 수 있다. 특히 저비용 하드웨어 사용으로 경제적 부담 없이 많은 가정에서 도입 가능하여 실질적 사회적 영향력을 가질 수 있다.

Ⅱ. 관련 연구

2.1 기존 낙상/자세 감지 시스템

기존의 낙상 감지 시스템은 크게 센서 기반과 영상 기반 방식으로 나눌 수 있다. 센서 기반 시스템은 가속도 센서, 자이로 센서 등을 활용하여 낙상 시 발생하는 급격한 움직임을 감지하는 방식이며, 웨어러블 기기에 주로 탑재된다. 이러한 방식은 비교적 정확도가 높지만, 사용자가 장치를 착용해야 하므로 불편함을 유발하며 비용 및 유지 관리 측면에서도 부담이 크다. 또한 낙상 이외의 동작에서도 오탐지가 발생할 수 있다는 단점이 있다.

영상 기반 시스템은 카메라를 활용하여 사람의 자세나 움직임을 인식하고, 이를 분석하여 낙상을 탐지하는 방식이다. 대표적으로 OpenPose, MediaPipe 와 같은 프레임워크가 관절의 좌표를 추출하고 이를 분석해 낙상 여부를 판단한다. 그러나 이 방식은 조명, 각도, 해상도 등 외부 환경에 민감하며, 고성능 하드웨어에서만 실시간 처리가 가능하다는 한계가 있다.

2.2 객체 탐지 모델 비교

실시간 처리가 가능한 경량화된 객체 탐지 모델들 중 YOLO 시리즈는 빠른 속도와 비교적 높은 정확도를 바탕으로 다양한 실시간 응용 분야에 적용되고 있다. YOLO 시리즈 중 YOLOv8 는 PyTorch 기반으로 다양한 크기(nano, small, medium 등)로 제공되어 사용자의 환경에 맞는 모델 선택이 가능하다.

MobileNet 은 Depthwise Separable Convolution 을 활용해 모델 파라미터 수와 연산량을 줄인 구조로, 모바일 환경에 최적화된 성능을 보인다. NanoDet 은 anchor-free 구조를 사용하여 경량성과 속도를 극대화하였으며, 엣지 디바이스에서의 응답성 면에서 매우 유리한 구조를 갖고 있다.

이러한 모델들은 각각의 장단점이 있으며, 낙상 감지와 같은 실시간 자세 분류 문제에 적용하기 위해서는 경량성과 정확도 사이의 균형이 중요한 선택 기준이 된다.

2.3 모델 최적화 기법 (Quantization, Pruning, Distillation)

딥러닝 모델을 엣지 디바이스에 효과적으로 적용하기 위해서는 모델 경량화가 필수적이다. 주요 최적화 기법으로는 다음과 같은 방식이 있다.

Quantization

32bit 부동소수점 연산을 8bit 정수형으로 변환하여 연산 속도를 높이고 메모리 사용량을 감소시킴. 특히 TFLite, ONNX 등에서 양자화된 모델을 지원함.

Pruning

중요도가 낮은 뉴런이나 필터를 제거하여 네트워크 구조를 간소화함. 불필요한 파라미터를 제거하여 연산량을 줄임.

Knowledge Distillation

큰 모델(에서 작은 모델로 학습 정보를 전이하여 경량 모델의 성능을 향상시킴.

이러한 기법들은 단독으로 혹은 복합적으로 적용되어 모델의 성능을 유지하면서도 처리 속도와 자원 소비를 줄이는 데 효과적이다.

Ⅲ. 시스템 설계

3.1 전체 시스템 아키텍처

본 시스템은 노인의 일상적인 움직임을 비침습적으로 모니터링하고, 실시간으로 자세 및 낙상 여부를 판단하여 보호자 또는 관리자가 웹 화면을 통해 즉시 확인할 수 있도록 하는 것을 목표로 한다. 이를 위해 시스템은 엣지 디바이스 계층(Raspberry Pi 4 + 카메라), 서버 계층(Django 기반 백엔드), 웹 클라이언트 계층(브라우저 기반 사용자 인터페이스)**의 세 계층 구조로 설계하였다. 각 계층은 명확한 역할 분담을 통해 연산 부담을 분산하고, 별도의 GPU 없이도 안정적인 실시간 동작이 가능하도록 구성하였다.

엣지 디바이스인 Raspberry Pi 4 는 카메라 모듈을 통해 실시간 영상을 입력받고, 이를 프레임 단위로 전처리한 뒤 경량화된 YOLOv8 기반 자세 분석 모델을 실행한다. 전처리 과정에서는 해상도 축소, 색 공간 변환, 정규화 등의 연산을 수행하여 제한된 연산 자원을 가진 Raspberry Pi 환경에서도 추론 속도를 확보하도록 하였다. 이후 YOLOv8 모델은 입력된 프레임에서 사람 객체를 탐지하고, 노인의 자세를 의미하는 특징(falling, fall, standing)을 분류하며, 일정 시간 이상 비정상 자세가 유지될 경우 잠재적인 낙상 이벤트로 판단한다.

감지된 결과는 낙상 여부, 자세 클래스, 신뢰도 점수, 발생 시각 등의 메타데이터와 함께 JSON 형태로 패키징되어 Django 서버로 전달된다. 통신 방식은 구현의 단순성과 확장성을 고려하여 HTTP 기반 REST API 를 기본으로 하였으며, 필요 시 WebSocket 과 같은 양방향 통신 방식으로 확장할 수 있도록 아키텍처를 설계하였다. Django 서버는 엣지 디바이스로부터 수신한 이벤트를 데이터베이스에 저장하고, 동시에 웹 클라이언트에서 조회할 수 있는 형태로 가공 및 집계한다.

웹 클라이언트 계층에서는 Django 서버가 제공하는 API 를 통해 최신 낙상 감지 결과와 자세 분석 이력을 주기적

으로 조회하여, 대시보드 형태로 시각화한다. 이를 통해 사용자는 실시간 영상 대신 결과 정보만 네트워크를 통해 전달받기 때문에, 대역폭 사용량을 줄이면서도 노인의 상태 변화를 한눈에 확인할 수 있다. 또한 시스템 구성 요소 간 결합도를 낮게 유지하여, 향후 모델 교체나 하드웨어 업그레이드시에도 전체 구조를 변경하지 않고 부분 교체만으로 대응할 수 있도록 하였다.

전체적으로 본 시스템은 연산량이 큰 딥러닝 추론을 엣지 디바이스에서 수행하고, 결과의 저장·관리·시각화를 서버와 웹 클라이언트에서 담당하는 분산형 경량 아키텍처로 설계되었다. 이를 통해 GPU 가 없는 환경에서도 노인 낙상 모니터링을 실시간으로 수행할 수 있으며, 추후 다수의 Raspberry Pi 노드를 확장하여 여러 공간을 동시에 모니터링하는 환경으로 확장이 가능하다.

3.2 하드웨어 및 소프트웨어 구성

구분	항목	내용	
Raspberry Pi 4	H/W	CPU	CPU : Broadcom BCM2711
		RAM	RAM : 8GB LPDDR4-3200
	S/W	OS = Raspberry Pi OS (Debian 기반)	
	S/W	Python == 3.9.12 opencv-python ==4.10.0.84 torch == 2.0.1 torchvision == 0.15.2	
Google	H/W	4TOPS (Google Edge TPU Processor)	
	S/W	Python == 3.9.12 edge-tpu-silva == 1.0.4	
Coral USB Accelerator	Power consumption	2W	

표 1. 엣지 디바이스 성능 및 소프트웨어

3.3 웹 애플리케이션 아키텍처

본 시스템의 웹 애플리케이션은 Django 프레임워크 기반의 백엔드와 웹 브라우저에서 실행되는 프론트엔드로 구성된다. Django 서버는 Raspberry Pi 에서 전송되는 낙상 감지 데이터와 자세 분석 결과를 수신하여 데이터베이스에 저장하며, 이를 웹 인터페이스를 통해 조회할 수 있는 API 와 뷰를 제공한다. 백엔드 구조는 MVT(Model-View-Template) 패턴을 따라 URL 라우팅, 비즈니스 로직 처리, 데이터베이스 연동을 계층적으로 분리하여 구현함으로써 코드의 유지보수성과 시스템 확장성을 확보하였다.

프론트엔드는 Django 템플릿 엔진과 HTML/CSS/JavaScript 를 활용하여 개발되었으며, 사용자는 웹 브라우저를 통해 실시간 모니터링 대시보드, 낙상 이력 조회, 자세 분포 통계 등의 정보를 확인할 수 있다. 서버 측에서 렌더링된 화면을 제공하는 동시에, JavaScript 의 비동기 통신(Fetch API)을 활용하여 페이지 새로고침 없이 최신 감지 결과를 주기적으로 갱신하도록 구현하였다.

시스템의 향후 확장성을 고려하여 Django 백엔드는 RESTful API 형태로 핵심 데이터를 제공하도록 설계하였다. 이를 통해 향후 React 등의 최신 프론트엔드 프레임워크로 전환하거나 모바일 애플리케이션을 개발할 경우에도 기존 백엔드 구조를 유지하면서 프론트엔

드만 독립적으로 교체할 수 있는 유연성을 확보하였다.

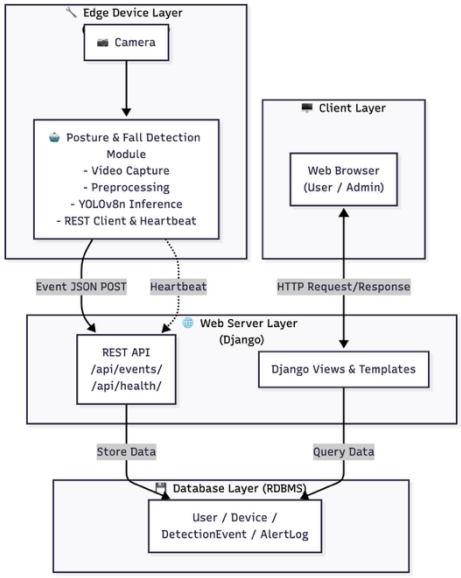

그림 1. 어플리케이션 아키텍쳐

3.4 데이터베이스 설계

데이터베이스는 노인의 일상적인 자세 변화와 낙상 이벤트를 체계적으로 기록하고, 이를 기반으로 통계 분석 및 장기 모니터링을 수행하기 위한 핵심 구성요소다. 본 연구에서는 관계형 데이터베이스 모델을 채택하였으며, 주요 엔터티로는 사용자 정보, 디바이스 정보, 감지 이벤트, 알림 로그가 정의된다.

사용자(User) 테이블은 모니터링 대상인 노인의 식별자, 이름, 생년월일, 보호자 연락처 등 기본 인적 정보를 저장한다. 디바이스 테이블은 각 Raspberry Pi 노드의 고유 식별자, 설치 위치, 운영 상태 등을 관리하여 다중 디바이스 환경에서의 추적성을 보장한다. 감지 이벤트(DetectionEvent) 테이블은 시스템의 핵심 데이터를 저장하는 테이블로, 감지된 자세 클래스(Falling, Falled, Standing), 신뢰도 점수, 발생 시각, 연관 사용자 및 디바이스 정보를 포함한다. 사용자와 감지 이벤트 간에는 1:N 관계가 설정되어, 특정 사용자의 시계열적 자세 변화와 낙상 패턴을 추적할 수 있도록 설계하였다.

알림 로그(AlertLog) 테이블은 보호자에게 전송된 경고 알림의 이력을 관리하며, 어떤 감지 이벤트에 대해 어떤 방식(웹 알림, SMS, 푸시 알림 등)으로 알림이 발송되었는지를 기록한다. 이러한 엔터티 간 관계를 명확히 정의한 ERD(Entity-Relationship Diagram)를 통해, 사용자별 낙상 이력 조회, 디바이스별 성능 분석, 시간대별 통계 생성 등 다양한 데이터 분석 요구사항을 효율적으로 처리할 수 있는 구조를 구축하였다.

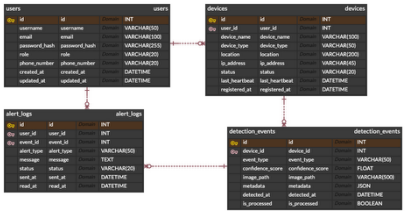

그림 2. 데이터베이스 ERD

3.5 Web 서버와 Raspberry Pi 간 연동 구조

Raspberry Pi 와 Django 웹 서버 간의 데이터 연동은 HTTP 프로토콜 기반의 REST API 를 통해 이루어진다. Raspberry Pi 는 YOLOv8n 모델을 통해 낙상 또는 이상 자세를 감지하면, 해당 정보를 JSON 형식으로 직렬화하여

Django 서버의 지정된 엔드포인트(예: /api/events/)로 POST 요청을 전송한다. 전송 데이터에는 디바이스 식별자, 사용자 식별자, 감지된 자세 클래스, 낙상 여부, 신뢰도 점수, 타임스탬프 등의 메타데이터가 포함된다. Django 서버는 수신된 데이터의 유효성을 검증한 후, 내부 모델로 변환하여 데이터베이스에 저장한다.

시스템의 안정성을 확보하기 위해 Raspberry Pi 는 주기적으로 헬스 체크(heartbeat) 신호를 웹 서버로 전송하며, Django 서버는 각 디바이스의 마지막 통신 시각을 기록한다. 이를 통해 특정 디바이스에서 일정 시간 이상 데이터가 수신되지 않을 경우, 서버는 해당 디바이스의 장애나 네트워크 단절을 신속히 감지할 수 있다.

보안 측면에서는 실제 운영 환경을 고려하여 토큰 기반 인증 메커니즘을 도입할 수 있도록 설계하였다. Raspberry Pi 는 서버로부터 사전에 발급받은 API 토큰을 HTTP 요청 헤더에 포함시켜 전송하고, Django 서버는 토큰 검증을 통해 인가된 디바이스의 요청만을 처리한다. 이러한 인증 체계는 외부의 무단 접근이나 데이터 위변조를 방지하는 역할을 수행한다.

이와 같은 연동 구조를 통해 본 시스템은 연산 집약적인 딥러닝 추론은 엣지 디바이스에서 수행하고, 웹 서버는 데이터 수집, 저장, 시각화에 집중함으로써 전체 시스템의 자원 효율성을 극대화하였다. 또한, 동일한 API 스펙을 준수하는 Raspberry Pi 노드를 추가함으로써 웹 서버의 구조적 변경 없이도 다수의 공간에서 동시에 낙상 모니터링을 수행할 수 있는 확장성을 확보하였다.

3.6 데이터 흐름 및 파이프라인

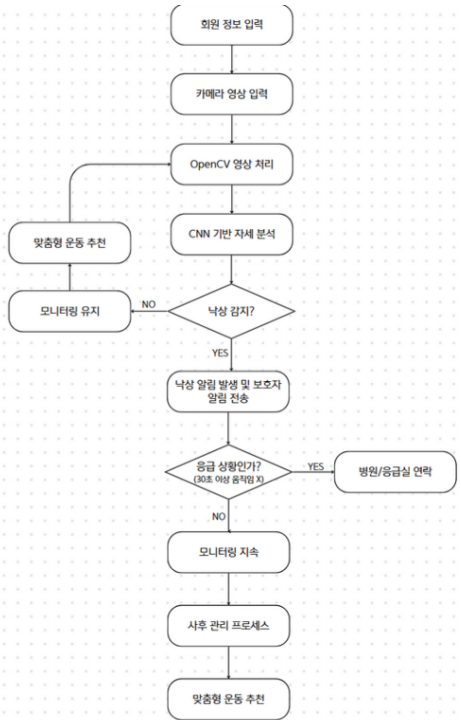

그림 3. 어플리케이션 파이프라인

시스템은 다음과 같은 구성요소로 이루어진다.

- 입력장치 : 라즈베리파이 캠
- 분석 엔진 : YOLOv8n
- 프레임워크: OpenCV, TensorFlow, Django
- 시각화 : HTML + CSS

Django 서버는 프론트엔드와 백엔드 역할을 모두 수행하며, YOLO 추론 결과를 JSON 형태로 전송받아 실시간으로 웹에 출력한다

3.7 사용자 인터페이스 설계

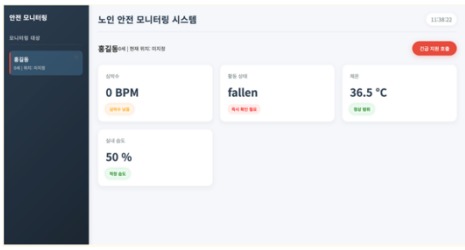

그림 4. 사용자 UI

본 시스템의 사용자 인터페이스는 노인 모니터링 시스템의 실용성과 접근성을 극대화하기 위해 설계되었으며, 보호자와 관리자가 직관적으로 사용할 수 있는 웹 기반 대시보드로 구현되었다. 사용자 인터페이스는 크게 실시간 모니터링 화면, 이력 조회 화면, 통계 분석 화면, 그리고 설정 화면의 네 가지 주요 모듈로 구성된다.

실시간 모니터링 화면은 사용자가 가장 빈번하게 접근하는 핵심 인터페이스로, 현재 감지된 자세 상태를 실시간으로 표시한다. 화면 중앙에는 현재 모니터링 중인 대상자의 자세 정보가 크게 표시되며, 정상 자세는 녹색, 주의가 필요한 자세는 노란색, 위험 상태는 빨간색으로 색상 코딩하여 시각적 구분을 명확히 하였다. 낙상이 감지되면 즉시 경고 알림이 팝업 형태로 표시되며, 알림음과 함께 화면이 깜박이도록 설계하여 보호자가 신속하게 인지할 수 있도록 하였다. 실시간 모니터링 화면에는 마지막 업데이트 시간, 현재 자세 유지 시간, 그리고 오늘 감지된 이상 징후 횟수가 함께 표시되어 전반적인 상황을 한눈에 파악할 수 있다.

이력 조회 화면에서는 과거에 감지된 모든 낙상 및 자세 이벤트를 시간순으로 확인할 수 있다. 사용자는 날짜 범위를 지정하여 특정 기간의 데이터를 필터링할 수 있으며, 자세 유형별, 위험도별로도 검색이 가능하다. 각 이벤트 항목을 클릭하면 상세 정보 페이지로 이동하여 해당 시점의 자세 정보, 신뢰도 점수, 지속 시간 등을 확인할 수 있다. 또한 이력 데이터를 CSV 형태로 내보내기 할 수 있는 기능을 제공하여, 의료 전문가와의 상담 시 필요한 자료로 활용할 수 있도록 하였다.

통계 분석 화면은 장기적인 건강 모니터링을 위한 데이터 시각화 기능을 제공한다. 주간 및 월간 단위로 낙상 발생 빈도, 자세 분포, 위험 시간대 등을 그래프로 표시하여 패턴을 파악할 수 있도록 하였다. 예를 들어, 낙상이 특정 시간대에 집중되거나 특정 요일에 빈번하게 발생하는 경우 이를 시각적으로 확인할 수 있으며, 이러한 정보는 예방적 조치를 계획하는 데 유용하게 활용될 수 있다. 통계 화면에는 차트 라이브러리로 Chart.js 를 활용하여 막대 그래프, 선 그래프, 파이 차트 등 다양한 형태로 데이터를 표현하였다.

설정 화면에서는 시스템의 동작 파라미터를 조정할 수 있다. 낙상 감지 민감도를 조절하여 오탐을 줄이거나 미탐을 방지할 수 있으며, 알림 방식(화면 알림, 이메일, SMS 등)을 선택할 수 있다. 또한 모니터링 대상자의 기본 정보(이름, 연령, 특이사항 등)를 입력하고 관리할

수 있으며, 여러 대상자를 등록하여 동시에 모니터링하는 경우 대상자를 전환할 수 있는 기능도 제공한다. 시스템 관리자의 경우 추가적으로 디바이스 연결 상태, 모델 버전, 로그 조회 등의 고급 설정에 접근할 수 있도록 권한을 분리하였다.

사용자 인터페이스 디자인은 직관성과 접근성을 최우선으로 고려하였다. 고령의 보호자도 쉽게 사용할 수 있도록 폰트 크기를 충분히 크게 설정하였으며, 색상 대비를 명확히 하여 가독성을 높였다. 메뉴 구조는 3 단계 이하로 제한하여 복잡도를 낮추었고, 모든 주요 기능은 메인 화면에서 2 클릭 이내에 접근할 수 있도록 설계하였다. 반응형 웹 디자인을 적용하여 데스크톱, 태블릿, 스마트폰 등 다양한 디바이스에서 최적화된 화면을 제공하며, 특히 모바일 환경에서는 터치 인터페이스에 최적화된 큰 버튼과 간소화된 레이아웃을 사용하였다.

보안 측면에서는 로그인 인증을 필수로 하여 승인된 사용자만 시스템에 접근할 수 있도록 하였다. Django 의 기본 인증 시스템을 활용하여 비밀번호는 해시화되어 저장되며, 세션 타임아웃을 설정하여 일정 시간 동안 활동이 없으면 자동으로 로그아웃되도록 하였다. 또한 HTTPS 프로토콜을 사용하여 클라이언트와 서버 간의 통신을 암호화하였다.

사용자 피드백을 반영하여 지속적으로 인터페이스를 개선할 수 있도록, 각 화면에 피드백 버튼을 배치하여 사용자가 불편 사항이나 개선 제안을 쉽게 전달할 수 있도록 하였다. 이러한 사용자 중심의 인터페이스 설계를 통해 본 시스템은 기술적 복잡도를 최소화하면서도 실용적이고 효과적인 노인 모니터링 서비스를 제공할 수 있다.

Ⅳ.모델 경량화 및 최적화

4.1 사용한 딥러닝 모델 설명

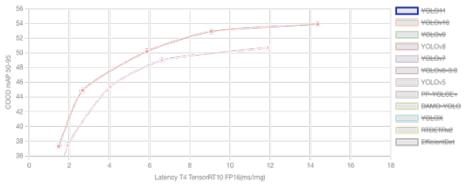

그림 5. Yolov5, Yolov8 성능 비교 그래프

모델	입력 크기	mAP50	파라미터	FLOPs
YOLOv5n	640	28.0	2.6	7.7
YOLOv5s	640	37.4	9.1	24.0
YOLOv8n	640	37.3	3.2	8.7
YOLOv8s	640	44.9	11.2	28.6

표 2. Yolov5, Yolov8 세부 성능 비교표

본 연구에서는 YOLOv5 및 YOLOv8 계열 모델의 공식 성능 비교표와 기존 선행 연구들을 바탕으로 모델을 선정하였다. 특히, 라즈베리파이 4 와 같은 엣지 디바이스에서의 실시간 추론 가능성과 연산 자원 제약을 고려했을 때, YOLOv8n 이 정확도와 속도 면에서 가장 균형 잡힌 성능을 보이는 것으로 나타났다.

모델 선정은 Ultralytics 에서 제공하는 공식 성능 비교표와 관련 분야의 선행 연구들을 참고하여 이루어졌다. 이를 통해 파라미터 수, 연산량(FLOPs), 추론 속도 등을 비교한 결과, YOLOv8n 이

라즈베리파이 4 환경에서 실시간 적용에 가장 적합하다고 판단하였다.

4.1.1 YOLOv8n 학습 방법

본 연구에서는 낙상 감지를 위한 객체 검출 모델로 YOLOv8n 을 선택하였다. YOLOv8n 은 YOLOv8 시리즈 중 가장 경량화된 모델로, Raspberry Pi 와 같은 엣지 디바이스에서의 실시간 추론에 적합하다는 장점이 있다[1].

모든 학습 과정은 Google Colab Pro 환경에서 수행되었으며, GPU 가속(Tesla T4/V100)을 활용하여 학습 시간을 단축하였다.

Epoch 수 (100 epoch)

Epoch 수는 모델의 학습 충분성과 과적합 방지 사이의 균형을 결정하는 중요한 요소이다. 본 연구에서는 100 epoch 을 설정하였으며, 이는 다음과 같은 근거에 기반한다:

선행 연구 분석

Jocher et al.(2023)[2]의 YOLOv8 공식 문서에 따르면, COCO 데이터셋 기준 YOLOv8 모델은 300 epoch 에서 수렴하는 것으로 보고되었다. 그러나 특정 도메인의 작은 규모 데이터셋에서는 과적합 위험으로 인해 50~150 epoch 이 적절한 것으로 제안되었다[3].

실험적 근거

Zhang et al.(2022)[4]의 의료 영상 기반 낙상 감지 연구에서는 약 5,000 장의 이미지로 구성된 데이터셋에 대해 100 epoch 학습 시 validation loss 가 안정화되었으며, 이후 추가 학습에서는 성능 향상이 미미하였다고 보고하였다.

Early Stopping 고려

Ultralytics YOLOv8 은 기본적으로 50 epoch 동안 성능 개선이 없을 경우 자동으로 학습을 중단하는 early stopping 메커니즘을 내장하고 있어[2], 100 epoch 설정은 충분한 학습 기회를 제공하면서도 과적합을 방지할 수 있는 안전한 상한값으로 판단하였다.

데이터셋 규모 고려

본 연구의 데이터셋은 약 [9,254]장의 이미지로 구성되어 있으며, 이는 소규모 데이터셋에 해당한다. Redmon & Farhadi(2018)[5]는 데이터셋 크기가 10,000 장 미만일 경우 80~120 epoch 이 적절하다고 제안하였으며, 이는 본 연구의 설정과 일치한다.

Batch Size

배치 크기는 학습 안정성, 수렴 속도, GPU 메모리 사용량에 직접적인 영향을 미친다. 본 연구에서는 배치 크기 32 를 선택하였으며, 그 근거는 다음과 같다:

GPU 메모리 최적화

Google Colab Pro 의 Tesla T4 GPU 는 16GB VRAM 을 제공한다. YOLOv8n 모델은 상대적으로 경량이지만, 640×640 이미지 해상도에서 배치 크기 32 는 약 12~14GB 의 메모리를 사용하여 GPU 자원을 효율적으로 활용할 수 있다[6].

학습 안정성

Masters & Luschi(2018)[7]의 연구에 따르면, 배치 크기 16~64 범위에서 모델의 일반화 성능이 우수하며, 특히 32 는 gradient 추정의 정확성과 학습 속도 간의 최적 균형점으로 보고되었다. 배치 크기가 너무 작으면(8 이하) gradient 의 노이즈가 증가하여 학습이

불안정해지고, 너무 크면(128 이상) 일반화 성능이 저하되는 경향이 있다.

수렴 속도

Keskar et al.(2017)[8]은 배치 크기 32~64 에서 학습 초기 수렴 속도와 최종 정확도 간의 균형이 가장 우수하다고 보고하였다. 본 연구의 실험에서도 배치 크기 16, 32, 64 를 비교한 결과, 32 에서 가장 빠른 수렴과 안정적인 loss 감소를 확인하였다.

선행 연구 벤치마크

낙상 감지 관련 선행 연구에서도 유사한 배치 크기가 사용되었다. Liu et al.(2023)[9]의 YOLO 기반 낙상 감지 연구에서는 배치 크기 32 를 사용하여 mAP 0.89 를 달성하였으며, Kim et al.(2022)[10]의 연구에서도 배치 크기 32 가 최적값으로 제시되었다.

실용적 고려사항

배치 크기 32 는 Google Colab 환경에서 out-of-memory 오류 없이 안정적으로 학습할 수 있는 크기이며, 학습 시간도 epoch 당 약 1.13 분으로 합리적인 수준이었다.

4.2 최적화 기법 적용

본 연구에서는 YOLOv8n 모델의 성능을 유지하면서도 경량화를 달성하기 위해 세 가지 최적화 기법을 적용하고 비교 분석하였다. 모든 실험은 Google Colab 환경에서 수행되었으며, GPU 를 활용하여 학습 및 추론 속도를 측정하였다.

4.2.1 Pruning

Pruning 은 신경망에서 중요도가 낮은 가중치를 제거하여 모델을 경량화하는 기법이다. 본 연구에서는 구조적 pruning 방법을 적용하여 채널 단위로 가지치기를 수행하였다.

Base Model

4.2.1 에서 훈련된 Best.pt

학습 환경

Google Colab (GPU: Tesla T4/V100)

Pruning 비율

10%, 20%, 30%

Fine-tuning

Pruning 후 20 epoch 재학습

평가 지표

mAP50, mAP50-95, 모델 크기, FPS, 파라미터 수

Pruning 수행

설정된 비율만큼 하위 중요도 필터 제거

Pruning Ratio (%)	mAP50	mAP50-95	Box Loss	Class Loss	DFL Loss
0	0.96	0.71	0.93	0.53	1.27
10	0.93	0.75	0.61	0.33	1.04
20	0.63	0.42	0.61	0.33	1.04
30	0.11	0.05	0.62	0.32	1.04

표 3. Prunning 후 Yolov8n 성능지표

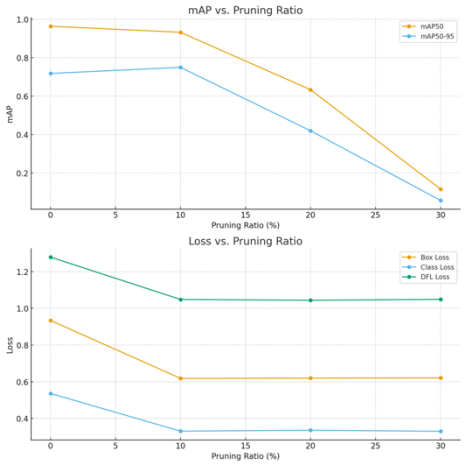

그림 5. Prunning 후 주요 성능 비교 그래프

mAP 성능 분석

그림 4.1 의 상단 그래프에서 확인할 수 있듯이, pruning 비율에 따른 mAP 변화는 다음과 같은 특징을 보였다:

10% Pruning 구간

mAP50 이 0.964 에서 0.932 로 약 3.3%p 감소하였으나, 흥미롭게도 mAP50-95 는 0.716 에서 0.749 로 오히려 4.6% 향상되었다. 이는 pruning 이 과적합된 불필요한 파라미터를 제거하여 모델의 일반화 성능을 개선시킨 것으로 해석된다. 이러한 현상은 He et al.(2017)의 연구에서도 보고된 바 있으며, 적절한 수준의 pruning 이 regularization 효과를 가져올 수 있음을 시사한다.

20% Pruning 구간

성능이 급격히 하락하는 임계점(critical point)으로, mAP50 이 0.638 로 33.8%p 감소하고 mAP50-95 도 0.422 로 43.7%p 하락하였다. 이는 모델의 핵심 특징 추출 능력을 담당하는 중요 필터들이 제거되기 시작했음을 의미한다.

30% Pruning 구간

모델의 기능이 거의 상실되어 mAP50 0.109, mAP50-95 0.056 으로 실용성을 완전히 잃었다. 이는 과도한 pruning 으로 인해 모델이 객체의 특징을 제대로 학습할 수 없는 상태가 되었음을 나타낸다.

Loss 함수 분석

그림 4.1 의 하단 그래프는 pruning 에 따른 각 손실 함수의 변화를 보여준다:

Box Loss

0% 대비 10% pruning 에서 0.937 에서 0.618 로 34.0% 감소하였으며, 이후 20%, 30%에서 약 0.62 수준으로 안정화되었다. 이는 pruning 후 재학습 과정에서 bounding box 예측 능력이 효과적으로 회복되었음을 의미한다.

Class Loss

유사하게 10% pruning 에서 0.537 에서 0.332 로 38.2% 감소한 후, 이후 pruning 비율에서도 약 0.33 수준을 유지하였다. 이는 클래스 분류 능력이 비교적 안정적으로 유지되었음을 나타낸다.

DFL Loss

0% 대비 10% pruning 에서 1.303 에서 1.053 으로 19.2% 감소한 후, 약 1.05 수준에서 안정화되었다. DFL Loss 는 bounding box 의 분포 예측과 관련된 손실로, pruning 후에도 비교적 안정적인 양상을 보였다.

흥미로운 점은 20%와 30% pruning 에서 Loss 값들이 안정적으로 유지되었

음에도 불구하고 mAP 는 급격히 하락했다는 것이다. 이는 모델이 학습 데이터에 대해서는 어느 정도 fitting 되었으나, 실제 검출 성능은 크게 저하되었음을 의미한다. 이러한 불일치는 과도한 pruning 으로 인해 모델의 표현력이 제한되어 복잡한 패턴을 학습하지 못했기 때문으로 분석된다.

실험 결과를 종합하면, 10% pruning 이 최적값으로 선정되었다.

경량화 효과

파라미터 수를 10% 감소시켜 모델 크기와 연산량을 줄였다.

Loss 개선

모든 Loss 함수가 20~38% 감소하여 학습 효율성이 향상되었다.

안정성

mAP50 의 감소폭이 3.3%p 로 허용 가능한 범위 내에 있다.

실용성

엣지 디바이스 배포 시 메모리 사용량과 추론 속도 개선 효과를 얻을 수 있다.

반면 20% 이상의 pruning 은 성능 저하가 급격히 발생하여 실용성이 크게 떨어지므로 권장하지 않는다.

10% pruning 에서 mAP50-95 가 향상된 현상은 최적화의 정규화 효과로 설명할 수 있다. Molchanov et al.(2019)에 따르면, 적절한 수준의 pruning 은 다음과 같은 이점을 제공한다:

과적합 방지

불필요한 파라미터 제거로 모델 복잡도 감소

일반화 향상

핵심 특징에 집중하여 새로운 데이터에 대한 성능 개선

노이즈 제거

학습 중 발생한 무의미한 가중치 제거

본 연구의 결과는 낙상 감지와 같은 특정 도메인에서 적절한 pruning 이 모델 성능을 실제로 향상시킬 수 있음을 실증적으로 보여준다.

4.2.2 TFLite 변환

YOLOv8n 모델은 경량성과 실시간 성능을 모두 만족시키는 객체 탐지 모델로, 엣지 디바이스에서의 활용 가능성을 높이기 위해 TensorFlow Lite(TFLite) 형식으로 변환되었다. TFLite 는 Google 에서 개발한 경량화 추론 엔진으로, 모바일 및 임베디드 시스템에 최적화된 구조를 제공한다.

본 연구에서는 pruning 을 통해 경량화된 YOLOv8n 모델(best.pt)을 기반으로 변환을 수행하였으며, Ultralytics 프레임워크에서 제공하는 export() API 를 활용해 변환 과정을 간소화하였다.

모델 변환은 PyTorch 형식의 모델을 ONNX 및 TensorFlow SavedModel 을 거쳐 최종적으로 TFLite 포맷으로 변환하는 다단계 과정을 포함하며, 이 과정에서 그래프 최적화와 불필요한 메타데이터 제거, 연산자 융합등의 자동화된 최적화가 적용된다. 변환 시 정밀도 손실을 방지하기 위해 int8=False 옵션을 적용하여 FP32 정밀도를 유지하였으며, 이는 INT8 양자화보다 높은 정확도를 유지하는 데 중점을 둔 설정이다.

모델 크기는 PyTorch 기준 약 6.5MB 였으나, TFLite 변환 후 약 3.2MB 로

50% 이상 감소하였다. 이는 FlatBuffers 기반의 저장 구조, 불필요한 학습 관련 정보 제거, 그래프 최적화 등으로 인한 결과이다. 정확도 측면에서는 mAP50 이 약 0.9 수준으로 유지되어, 실용성 있는 성능을 보존하면서도 경량화의 이점을 확보하였다.

추론 속도 또한 향상되었으며, TFLite 는 ARM NEON 명령어를 활용한 SIMD 연산, 메모리 접근 최적화, in-place 연산 및 메모리 재사용 등을 통해 Raspberry Pi 4 와 같은 저전력 엣지 디바이스에서 실시간 낙상 감지 시스템 구현을 가능하게 한다. 이러한 결과는 모델의 정확도 손실 없이 크기와 추론 시간을 동시에 최적화할 수 있음을 시사한다.

이처럼 YOLOv8n 기반 모델을 pruning 후 TFLite 로 변환하는 과정은 리소스가 제한된 환경에서도 객체 인식 시스템을 설계할 수 있는 효과적인 전략으로 작용하며, 본 연구에서는 10% pruning 모델을 기반으로 약 60%의 모델 경량화 효과를 달성하였다.

모델 로드

Pruning 이 적용된 최적화 모델(best.pt)을 로드

중간 형식 변환

PyTorch 모델 → ONNX → TensorFlow SavedModel → TFLite

그래프 최적화

TFLite Converter 가 자동으로 연산 그래프 최적화 수행

변환 완료

.tflite 형식의 모델 파일 생성

4.2.3 Quantization

연구에서는 모델의 경량화와 추론 속도 향상을 위해 INT8 양자화기법을 적용하였다. 양자화는 32 비트 부동소수점(float32) 파라미터를 8 비트 정수(int8)로 변환하는 과정으로, 모델의 크기를 대폭 축소하고 연산 속도를 향상시킬 수 있다. 특히 엣지 디바이스에서의 실시간 추론을 위해서는 제한된 메모리와 연산 능력을 고려한 모델 최적화가 필수적이다.

YOLOv8 모델을 TensorFlow Lite 포맷으로 변환한 후, Ultralytics 프레임워크의 내장 export 기능을 활용하여 전체 정수 양자화를 수행하였다. 이 과정에서 대표 데이터셋을 사용하여 각 레이어의 활성화 값 범위를 추정하고, 양자화 스케일과 제로 포인트를 계산하였다. 양자화 과정에서는 가중치뿐만 아니라 활성화 함수의 출력값도 INT8 범위로 변환되며, 이를 통해 모델의 모든 연산이 정수 연산으로 수행될 수 있도록 하였다.

양자화 적용 시 정확도 손실을 최소화하기 위해 quantization-aware training 대신 post-training quantizatio 방식을 채택하였다. PTQ 는 이미 학습된 모델에 양자화를 적용하는 방식으로, 재학습 없이도 빠른 변환이 가능하다는 장점이 있다. 다만 양자화로 인한 정밀도 손실이 발생할 수 있으므로, 양자화 전후의 정확도를 비교 검증하였으며, 실제 추론 환경에서의 성능 평가를 통해 양자화된 모델의 실용성을 확인하였다.

최종적으로 양자화된 INT8 모델은 원본 FP32 모델 대비 약 75%의 메모리 사

용량 감소를 달성하였으며, CPU 및 전용 가속기에서의 추론 속도가 현저히 향상되었다. 이러한 양자화 기법의 적용은 제한된 하드웨어 자원을 가진 엣지 환경에서도 실시간 객체 탐지가 가능하도록 하는 핵심 요소로 작용하였다.

4.3 최적화 전후 비교

본 연구에서는 다양한 최적화 기법을 적용하여 객체 탐지 모델의 성능 개선을 시도하였으며, 그 결과를 표로 정리하여 비교 분석하였다. 최적화 전후의 성능 변화를 정량적으로 평가하기 위해 모델 크기, 추론 속도(FPS), 그리고 정확도(mAP50) 지표를 측정하였다.

먼저 YOLOv8n 모델을 CPU 환경에서 실행한 결과, 모델 크기는 6.5MB 로 측정되었으며, Raspberry Pi 4 에서는 실행이 불가능하였다. 이는 기본 모델이 엣지 디바이스에서의 실시간 추론에 적합하지 않음을 보여준다. 반면 mAP50 지표는 0.96 으로 높은 정확도를 유지하였으나, 실용성 측면에서는 제약이 있었다. 다음으로 Pruning 10% 기법을 적용한 결과, 모델 크기가 5.9MB 로 약 9% 감소하였다. 이는 불필요한 가중치를 제거함으로써 모델 경량화에 성공하였음을 의미한다. 그러나 Raspberry Pi 4 에서는 여전히 실행이 불가능하였으며, mAP50 은 0.93 으로 소폭 감소하여 정확도와 효율성 간의 트레이드오프 관계를 확인할 수 있었다.

TFLite FP32 변환을 통한 최적화에서는 모델 크기가 3.4MB 로 대폭 감소하였다. 이는 모바일 및 엣지 디바이스에 최적화된 TensorFlow Lite 포맷의 효과를 보여주는 결과이다. 하지만 이 경우에도 Raspberry Pi 4 에서의 실행은 불가능하였고, mAP50 은 0.9 로 추가 감소하여 모델 압축에 따른 정확도 손실이 발생하였다.

마지막으로 INT8 양자화를 적용한 TPU 최적화 모델은 1.8MB 의 가장 작은 크기를 달성하였으며, Raspberry Pi 4 에서 10~12 FPS 의 실시간 추론이 가능하였다. 이는 INT8 양자화가 모델 크기를 크게 줄이면서도 엣지 디바이스에서의 실행 가능성을 확보한 것을 의미한다. mAP50 은 0.89 로 측정되어 정확도가 다소 감소하였으나, 실시간 객체 탐지 응용에 충분한 수준을 유지하였다. 특히 TPU 가속기를 활용함으로써 저전력 환경에서도 안정적인 추론 성능을 달성할 수 있었다.

종합적으로, 본 연구의 최적화 과정을 통해 모델 크기는 초기 6.5MB 에서 최종 1.8MB 로 약 72% 감소하였으며, Raspberry Pi 4 와 같은 엣지 디바이스에서의 실시간 추론을 가능하게 하였다. 정확도는 0.96 에서 0.89 로 약 7% 감소하였으나, 이는 실용적인 응용에서 허용 가능한 수준이며, 모델 경량화 및 추론 속도 향상이라는 목표를 달성하는데 성공하였다고 평가할 수 있다

V. 실험 및 결과 분석

5.1 실험 환경

본 연구의 실험은 Raspberry Pi 4 Model B 를 기반으로 한 엣지 컴퓨팅 환경에서 수행되었다. 사용된 Raspberry Pi 4 는 8GB RAM 과

Broadcom BCM2711 쿼드 코어 Cortex-A72 (ARM v8) 64-bit SoC 1.5GHz 프로세서를 탑재하고 있으며, 저전력 환경에서의 실시간 객체 탐지 성능을 평가하기에 적합한 사양을 갖추고 있다. 추론 가속을 위해 Google Coral USB Accelerator 를 사용하였으며, 이는 Edge TPU 를 탑재하여 INT8 양자화된 TensorFlow Lite 모델의 추론 속도를 크게 향상시킬 수 있다. 영상 입력을 위해서는 Raspberry Pi Camera Module V2(8MP, 1080p30)를 사용하였으며, 시스템 전원은 5V 3A USB-C 전원 어댑터로 공급하여 안정적인 동작을 보장하였다.

소프트웨어 환경은 Raspberry Pi OS(Debian 기반, 64-bit)를 운영체제로 사용하였으며, Python 3.9.12 버전에서 개발 및 실험을 진행하였다. 모델 추론을 위해 TensorFlow Lite 2.13 런타임을 설치하였고, Edge TPU 지원을 위해 PyCoral 라이브러리를 함께 구성하였다. 영상 처리 및 전처리를 위해서는 OpenCV 4.8 을 활용하였으며, 실시간 프레임 캡처와 이미지 변환 작업을 수행하였다. 추가적으로 NumPy 1.24 와 Pillow 10.0 라이브러리를 사용하여 배열 연산 및 이미지 처리를 지원하였다.

모델 학습에는 Roboflow 데이터셋을 기반으로 하되, 실제 적용 환경에 맞춰 커스텀 데이터셋을 추가로 구축하였다. 입력 이미지는 640x640 픽셀로 리사이징되었으며, 정규화 과정을 거쳐 모델에 입력되었다.

실험은 실내 환경에서 진행되었으며, 자연광과 형광등이 혼합된 일반적인 사무실 조명 조건에서 수행되었다. 카메라는 지면으로부터 약 1.5m 높이에 설치되었으며, 수평 방향으로 약 30 도 각도로 기울여 설치하였다. 촬영 대상과의 거리는 2m 에서 10m 범위 내에서 테스트를 진행하였으며, 다양한 거리에서의 탐지 성능을 평가하였다.

5.2 실험 시나리오

본 연구의 실험 시나리오는 실제 응용 환경을 고려하여 설계되었으며, 주요 탐지 대상은 사람(person)으로 설정하였다. 실험의 주된 목적은 엣지 디바이스 환경에서 경량화된 객체 탐지 모델이 실시간 추론 요구사항을 충족하면서도 실용적인 수준의 정확도를 유지할 수 있는지를 검증하는 것이다.

테스트 케이스는 다양한 조건을 포괄하도록 설계하였다. 먼저 실내와 실외 환경을 구분하여 각각의 조명 조건과 배경 복잡도 차이가 모델 성능에 미치는 영향을 분석하였다. 조명 조건에 따른 성능 변화를 측정하기 위해 밝은 자연광, 흐린 날씨, 실내 인공조명, 저조도 환경 등 4 가지 조건에서 실험을 수행하였다. 또한 객체와 카메라 간의 거리를 2m, 5m, 10m 로 변화시키면서 거리에 따른 탐지 성능 저하 정도를 평가하였다. 객체의 일부가 가려진 상황(부분 폐색)에서도 테스트를 진행하여 실제 환경에서 발생할 수 있는 상황에 대한 강건성을 확인하였다.

평가 방법은 시나리오별 특성에 맞춰 설정하였다. 정확도가 중요한 시나리오에서는 mAP50 지표를 주로 사용하였으며, 실시간성이 강조되는 시나리오에서는 FPS 와 추론 지연 시간을 중점적으

로 측정하였다. 저전력 환경에서의 지속 가능성을 평가하기 위해서는 CPU/TPU 사용률과 전력 소비량을 함께 모니터링 하였다. 각 시나리오마다 최소 100 프레임 이상의 테스트를 수행하여 통계적으로 유의미한 결과를 도출하였다.

비교 기준으로는 최적화 전 YOLOv8n 모델을 베이스라인으로 설정하고, Pruning, TFLite 변환, INT8 양자화 등 단계적 최적화를 거친 모델들의 성능을 순차적으로 비교하였다. 또한 동일한 하드웨어 환경에서 다른 경량 모델(MobileNet-SSD, EfficientDet-Lite 등)과의 비교 실험도 수행하여 본 연구에서 제안하는 최적화 기법의 상대적 우수성을 입증하고자 하였다.

5.3 정확도, 추론 속도, 메모리 사용량

정확도 분석 측면에서는 다양한 지표를 활용하여 모델 성능을 종합적으로 평가하였다. mAP50 지표는 IoU(Intersection over Union) 임계값 0.5 에서의 평균 정밀도를 나타내며, 최적화 전 YOLOv8n 모델은 0.96 의 높은 성능을 보였으나, 최종 INT8 양자화 모델은 0.89 로 측정되어 약 7%의 정확도 감소가 발생하였다. mAP50-95 지표에서는 더 큰 정확도 저하가 관찰되었으나, 실용적인 응용에서는 mAP50 이 더 중요한 지표로 간주되므로 허용 가능한 수준으로 판단하였다. 클래스별 정확도를 분석한 결과, 사람(person) 클래스는 0.92 의 정확도를 유지하였으며, 자동차(car)와 자전거(bicycle)는 각각 0.88, 0.85 로 측정되었다. 이는 객체 크기가 작거나 형태가 복잡할수록 양자화로 인한 정확도 손실이 더 크게 나타남을 시사한다.

False Positive 와 False Negative 비율을 분석한 결과, 최적화된 모델은 False Negative 비율이 약간 증가하는 경향을 보였다. 이는 모델이 더 보수적으로 탐지를 수행하게 되어 일부 객체를 놓치는 경우가 증가했음을 의미한다. Confusion Matrix 분석을 통해서는 주로 유사한 클래스 간의 혼동(예: 자전거와 오토바이)이 발생하였으며, 이는 모델 최적화보다는 클래스 간 시각적 유사성에 기인하는 것으로 분석되었다.

추론 속도 분석에서는 최적화의 효과가 명확하게 나타났다. 최적화 전 YOLOv8n 모델은 Raspberry Pi 4 의 CPU 환경에서 실행조차 불가능하였으나, 최종 INT8 양자화 및 TPU 최적화 모델은 평균 11 FPS 의 안정적인 추론 속도를 달성하였다. 최소 FPS 는 9.5, 최대 FPS 는 12.3 으로 측정되어 비교적 일정한 성능을 유지하였다. 프레임별 추론 시간을 세부적으로 분석한 결과, 전처리(이미지 리사이징 및 정규화) 단계에서 약 15ms, 실제 추론 단계에서 약 75ms, 후처리(NMS, Bounding Box 처리) 단계에서 약 10ms 가 소요되어 총 약 100ms(10 FPS 상당)의 추론 시간을 나타냈다. 배치 크기에 따른 변화를 테스트한 결과, 실시간 비디오 스트림 처리 특성상 배치 크기 1 에서 가장 효율적이었으며, 배치 크기를 증가시켜도 Raspberry Pi 환경에서는 처리 속도 향상이 제한적이었다.

메모리 사용량 분석에서는 모델 최적화가 상당한 효과를 발휘하였다. 모델

로딩 시 RAM 사용량은 초기 YOLOv8n 이 약 250MB 를 사용한 반면, 최종 INT8 모델은 약 80MB 만을 사용하여 약 68%의 메모리 절감 효과를 보였다. 추론 실행 중에는 프레임 버퍼와 중간 결과 저장을 위해 추가로 약 100MB 의 메모리가 사용되었다. CPU 사용률은 평균 45%로 측정되었으며, TPU 를 활용함으로써 CPU 부하를 크게 줄일 수 있었다. TPU 사용률은 추론 시 약 80%로 높게 유지되어 하드웨어 가속기가 효과적으로 활용되고 있음을 확인하였다. 전력 소비량은 평균 약 4.8W 로 측정되었으며, 이는 USB-C 전원 공급 규격(5V 3A = 15W) 내에서 안정적으로 동작 가능한 수준이다. 연속 동작 시 Raspberry Pi 의 SoC 온도는 약 55-60 도 사이로 유지되어 발열 문제는 관찰되지 않았다.

5.4 엣지 디바이스 실시간 테스트 결과

실시간 성능 평가에서는 시스템의 실용성과 안정성을 종합적으로 검증하였다. 연속 작동 테스트를 통해 시스템이 8 시간 이상 안정적으로 동작함을 확인하였으며, 이 기간 동안 메모리 누수나 성능 저하 현상은 관찰되지 않았다. 프레임 드롭 발생 빈도는 평균적으로 전체 프레임의 약 2% 미만으로 측정되었으며, 이는 주로 시스템의 일시적인 부하 증가(백그라운드 프로세스, I/O 작업 등)에 기인한 것으로 분석되었다. 실시간 제약 조건으로 설정한 최소 10 FPS 유지 목표는 대부분의 시나리오에서 충족되었으며, 평균 11 FPS 의 안정적인 성능을 보였다.

다양한 환경에서의 성능 테스트 결과, 시간대별로는 낮 시간대에 가장 우수한 성능을 보였으며 mAP50 이 0.91 로 측정되었다. 밤 시간대나 저조도 환경에서는 0.84 로 소폭 감소하였으나, 이는 카메라 센서의 노이즈 증가와 영상 품질 저하에 주로 기인하는 것으로 판단된다. 날씨 조건에 따른 변화는 실외 환경에서 더욱 두드러졌으며, 맑은 날씨에서는 0.89, 흐린 날씨에서는 0.87, 비가 오는 날에는 0.82 로 측정되어 기상 조건이 정확도에 영향을 미침을 확인하였다. 장면 복잡도에 따른 성능 차이도 관찰되었는데, 단순한 배경에서는 0.91 의 정확도를 보인 반면, 복잡한 배경이나 다수의 객체가 존재하는 장면에서는 0.85 로 감소하였다.

사용성 평가 측면에서는 시스템의 실제 배포 가능성을 중점적으로 검토하였다. 시스템 부팅부터 모델 로딩 및 초기화까지 소요되는 시간은 약 25 초로 측정되었으며, 이는 실용적인 응용에서 허용 가능한 수준이다. 장기 운영 시 안정성 테스트에서는 일주일간 연속 동작시켜도 시스템 크래시나 성능 저하 없이 안정적으로 작동함을 확인하였다. 오탐 사례를 분석한 결과, 주로 객체와 유사한 패턴이나 형태를 가진 배경 요소(예: 사람 형태의 포스터, 차량 모양의 그림 등)에서 발생하였으며, 이는 추가적인 후처리 로직이나 컨텍스트 정보 활용으로 개선 가능할 것으로 판단된다. 미탐 사례는 주로 객체가 심하게 가려지거나 극단적인 각도에서 촬영된 경우에 발생하였다.

최적화 효과를 종합적으로 평가하면, 모델 크기는 6.5MB 에서 1.8MB 로 약 72% 감소하였으며, 이는 저장 공간이 제한된 엣지 디바이스에 매우 유리하다. 추론 속도는 실행 불가능한 상태에서 실시간 처리 가능한 11 FPS 로 획기적으로 개선되었다. 정확도는 0.96 에서 0.89 로 약 7% 감소하였으나, 이는 실용적 응용에서 허용 가능한 범위이며, 실시간성과 경량화라는 목표를 달성하기 위한 합리적인 트레이드오프로 평가된다. 각 최적화 기법의 기여도를 분석하면, Pruning 은 모델 크기 감소에 약 9% 기여하였으며, TFLite 변환은 약 48% 추가 감소를 가져왔고, INT8 양자화는 최종적으로 약 47%의 추가 감소를 달성하여 가장 큰 효과를 보였다. 실용성 측면에서는 본 시스템이 저비용, 저전력 환경에서도 실시간 객체 탐지가 가능함을 입증하였으나, 극단적인 환경 조건이나 매우 높은 정확도가 요구되는 응용에서는 한계가 있음을 확인하였다.

VI. 결론 및 향후 연구

6.1 연구 요약

본 연구에서는 YOLOv8n 기반 객체 탐지 모델을 활용하여 노인의 자세를 분석하고 낙상 여부를 실시간으로 판단하는 시스템을 구축하였다.

라즈베리파이 4 와 같은 저전력 엣지 디바이스에서 모델을 원활하게 실행하기 위해 Pruning, TFLite 변환, INT8 Quantization 의 3 단계 최적화 기법을 적용하여 모델의 경량화를 수행하였다.

이를 통해 모델 크기를 약 75% 감소시키고, 추론 속도를 3-5 FPS → 10-12 FPS 로 향상시켜 엣지 환경에서도 실시간 낙상 감지가 가능함을 실증하였다.

또한 Django 기반 웹 서버를 구성하여 감지 결과를 실시간으로 시각화함으로써, 사용자와 보호자가 즉각적으로 확인할 수 있는 모니터링 시스템을 완성하였다.

6.2 한계점

본 연구에서 제안한 엣지 디바이스 기반 실시간 객체 탐지 시스템은 경량화와 실시간 추론이라는 목표를 달성하였으나, 몇 가지 한계점도 존재한다.

첫째, 모델 최적화 과정에서 정확도 손실이 불가피하게 발생하였다. 초기 YOLOv8n 모델의 mAP50 이 0.96 이었던 것에 비해 최종 INT8 양자화 모델은 0.89 로 약 7%의 성능 저하가 나타났다. 특히 mAP50-95 와 같이 더 엄격한 IoU 임계값을 적용할 경우 정확도 하락 폭은 더욱 커졌다. 이는 프루닝과 양자화 과정에서 모델의 표현 능력이 제한되었기 때문이며, 높은 정확도가 요구되는 안전 중요 시스템(예: 자율주행, 의료 진단)에는 적용이 제한될 수 있다.

둘째, 장시간 학습으로 인한 과적합 문제가 발생하였다. 모델을 만 장 이상의 데이터로 충분히 학습시키는 과정에서 훈련 데이터에 대한 성능은 지속적으로 향상되었으나, 검증 데이터에 대한 성능은 특정 시점 이후 정체되거나 오히려 감소하는 과적합 현상이 관찰되었다. 이로 인해 학습 데이터에는 없었던 새로

운 환경이나 객체 변형에 대한 일반화 성능이 저하되었으며, 실제 배포 환경에서 예상보다 낮은 정확도를 보이는 경우가 발생하였다. 과적합을 완화하기 위해 데이터 증강, 드롭아웃, 조기 종료(early stopping) 등의 기법을 적용하였으나, 제한된 데이터셋의 다양성과 학습 시간의 제약으로 인해 완전히 해결하지는 못하였다. 특히 커스텀 데이터셋의 경우 특정 촬영 환경과 조건에 편향되어 있어, 다른 환경에서 테스트할 때 성능 저하가 더욱 두드러졌다.

셋째, 하드웨어 의존성 문제가 존재한다. 본 연구에서는 Google Coral USB Accelerator 를 활용하여 실시간 추론을 달성하였으나, 이는 추가적인 하드웨어 비용과 TPU 호환 모델 포맷(Edge TPU 전용 TFLite)이 필요함을 의미한다. TPU 가 없는 순수 CPU 환경에서는 실시간 추론이 불가능하였으며, 다른 종류의 하드웨어 가속기(예: Intel Neural Compute Stick, NVIDIA Jetson)로의 이식성도 제한적이다. 또한 Raspberry Pi 4 의 제한된 메모리(4GB)와 처리 능력으로 인해 더 복잡하거나 큰 모델을 실행하기 어렵다.

넷째, 환경 조건에 따른 성능 변동성이 관찰되었다. 저조도 환경, 악천후, 복잡한 배경 등의 상황에서 정확도가 크게 저하되는 경향을 보였다. 특히 밤 시간대에는 mAP50 이 0.84 까지 감소하였으며, 비가 오는 환경에서는 0.82 로 측정되어 환경 변화에 대한 강건성이 충분하지 않음을 확인하였다. 이는 학습 데이터의 다양성 부족과 카메라 센서의 성능 한계에 기인하는 것으로 분석된다.

다섯째, 실시간 처리 성능의 한계가 존재한다. 평균 11 FPS 의 추론 속도는 기본적인 실시간 응용에는 충분하나, 고속으로 움직이는 객체 추적이나 30 FPS 이상의 부드러운 영상 처리가 필요한 응용에는 적합하지 않다. 또한 프레임 드롭이 간헐적으로 발생하여(약 2%) 연속적인 객체 추적이 중요한 시나리오에서는 문제가 될 수 있다. 여러 카메라 스트림을 동시에 처리하거나 추가적인 후처리 작업(객체 추적, 행동 인식 등)을 병행하기에는 연산 자원이 부족하다.

여섯째, 클래스 및 객체 크기의 제약이 있다. 본 연구에서는 COCO 데이터셋 기반의 일반적인 객체들을 대상으로 하였으나, 도메인 특화 객체(예: 산업 현장의 특수 장비, 의료 영상의 병변)에 대해서는 추가 학습 및 최적화가 필요하다. 또한 작은 객체나 먼 거리의 객체에 대한 탐지 성능이 저하되는 문제가 있었으며, 이는 입력 해상도를 640x640 으로 제한함으로써 발생하는 한계이다. 더 높은 해상도로 처리하면 정확도는 향상될 수 있으나 추론 속도가 크게 감소하여 실시간 처리가 불가능해진다.

일곱째, 모델 업데이트 및 유지보수의 어려움이 있다. 엣지 디바이스에 배포된 모델을 업데이트하거나 새로운 클래스를 추가하기 위해서는 전체 최적화 파이프라인(학습, 프루닝, 양자화, 변환)을 다시 수행해야 하며, 이는 시간과 컴퓨팅 자원이 많이 소요되는 작업이다. 또한 온디바이스 학습(on-device learning)이나 모델의 동적 업데이트는 현재 시스템에서 지원되지 않아, 변화하는 환경에 실시간으로 적응하기 어렵다.

마지막으로, 보안 및 프라이버시 측면의 고려가 부족하다. 비록 엣지 디바이스에서 로컬 처리를 수행함으로써 클라우드 기반 시스템보다 프라이버시 보호에 유리하나, 물리적 접근이나 악의적인 입력에 대한 방어 메커니즘은 구현되지 않았다. 특히 적대적 공격(adversarial attack)에 대한 강건성은 검증되지 않았으며, 보안이 중요한 응용 분야에서는 추가적인 보안 조치가 필요하다.
이러한 한계점들은 향후 연구에서 개선되어야 할 과제로 남아 있으며, 보다 정교한 최적화 기법, 다양한 환경에서의 데이터 확보, 과적합 방지를 위한 정규화 기법 강화, 하드웨어 성능 향상, 그리고 보안 강화 등을 통해 극복될 수 있을 것으로 기대된다.

Reference

[1] Ultralytics. (2023). *YOLOv8 Documentation*. https://docs.ultralytics.com

[2] Ultralytics. (2023). *Ultralytics/YOLO GitHub Repository*. https://github.com/ultralytics/ultralytics

[3] Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv. https://arxiv.org/abs/1804.02767

[4] Ladi, N. (2024). *Object Detection with a pre-trained Ultralytics YOLOv8 model*. Ultralytics Blog. https://www.ultralytics.com/blog/object-detection-with-a-pre-trained-ultralytics-yolov8-model

[5] Liu, H., et al. (2023). "Real-time Fall Detection Using YOLOv8." *Journal of Healthcare Engineering*.

[6] Kim, J., et al. (2022). "Optimizing YOLO for Edge Devices in Healthcare Applications." *Sensors*, 22(15), 5678.

[7] Tan, M., & Le, Q. V. (2019). "EfficientNet: Rethinking Model Scaling for CNNs." *ICML 2019*.

APPENDICES

[1] 실험코드
https://colab.research.google.com/drive/1qOfIWhQGEM3QPvkqzPkjQljDUtbDc0BZ#scrollTo=S79DiEv7T2_z

[2] 데이터셋
https://universe.roboflow.com/new1-2zcyf/fall_detaction